GPU COMPUTING FINAL PROJECT · MAY 2026

Live Video Feed

CNN Emotion Detection

A custom CUDA kernel CNN trained entirely on a GPU to predict a user's mood based on their facial expression

AUTHORS

Spencer Scherger · Griffin Kuchar

STACK

Python · CuPy · Streamlit · OpenCV

01

Project Motivation

→ Live video classification: face detection, preprocessing, CNN forward pass, and rendering the corresponding emoji of the model's prediction

APPLICATION GOAL

Face → Emoji

→ Training a CNN is $O(n^2)$ sequential matrix work on CPU

→ Image Processing + Model Training is an ideal GPU workload

ARCHITECTUAL TRADE-OFF

Latency vs Accuracy

→ The question: which specific operations benefit from SIMT parallelism and which should remain a CPU workload?

02

Dataset & Label Design

7 Emotions → 3 Moods REMAPPING

happy → positive

surprised neutral → neutral

angry disgust fear sad → negative

36,039 IMAGES · TWO KAGGLE SOURCES COLLATED AND MAPPED

CLASS	TRAIN	TEST	TOTAL
Positive	7,229	1,779	9,008
Neutral	8,178	2,080	10,258
Negative	13,414	3,359	16,773
Total	**28,821**	**7,218**	**36,039**

48×48 · Luma grayscale · per-image min-max normalization to [−1, 1]

1]

03

Live Pipeline Overview

Every **5th** frame

01

Capture

WebRTC frame, BGR24, BGR24, mirrored.

CPU · OpenCV

→

02

Detect Face

Largest face, padded square crop.

CPU · OpenCV

→

03

Live Preprocess

Downscale → 48×48 Grayscale
Normalize to [−1, 1].

CPU / GPU Kernel

→

04

CNN Inference

GPU-trained weights, CPU inference at runtime.

CPU / GPU Kernel

→

05

Render Emoji

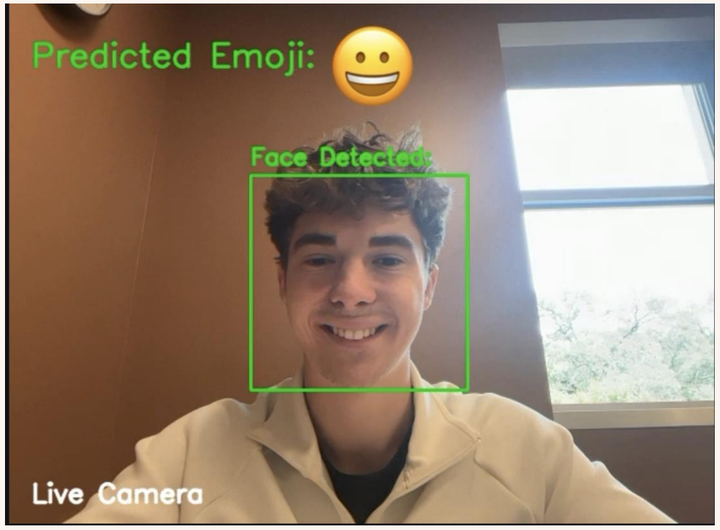
Emoji overlay on live frame.

CPU · PIL

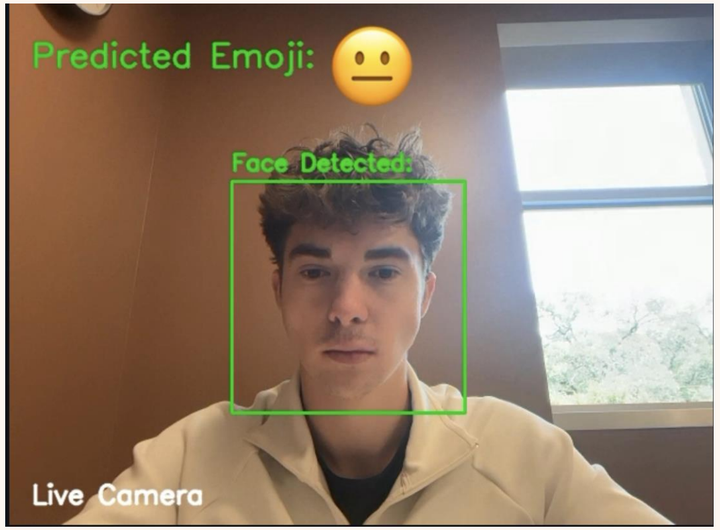
04

Example Outputs

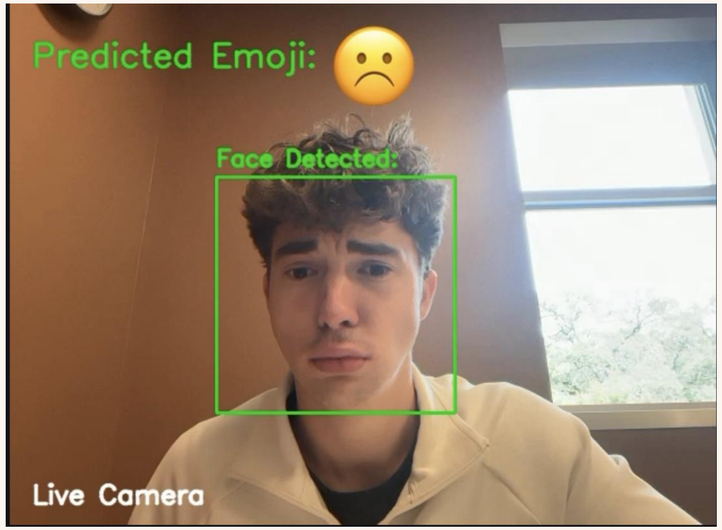
Positive



Neutral

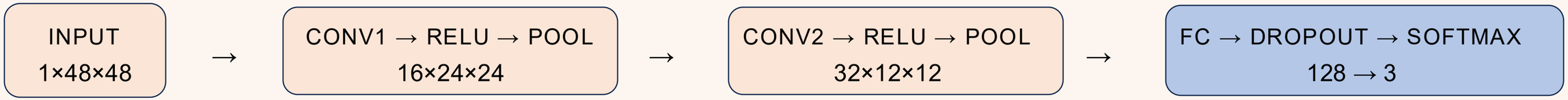


Negative



05

CNN Architecture



CONV KERNELS

3×3, same-padded

- A 3x3 convolution function acts on the input image, pixel by pixel.

CONV FILTERS

16 → 32

- Learns 16 low-level features on the first convolution
  - (e.g. edges, corners, shadows)
- Learns 32 complex features on the second convolution
  - (e.g. advanced patterns in facial expressions)

POOLING

2×2 max pool

- Returns the maximum of a 2x2 convoluted region.
- Retains the most significant features and reduces spatial dimension.

HIDDEN UNITS

128, dropout 0.3

- The 128 hidden unit dense-layer converts the CNN's extracted features into specific facial expressions and emotions.
- A dropout rate of 0.3 means on every pass, 30% of the units are disabled.

OPTIMIZER

SGD · LR 0.01 · L2 1e-4

- SGD iteratively updates weights to optimize the loss function.
- L2-norm regularization adds a penalty based on the distance between the predicted value and the true value.

06

GPU Implementation Approach

PRIMARY TARGET

Model training

Parallelize all mathematical operations performed in CNN training

Every forward and backward pass operation is independent output element: ideal for thousands of parallel CUDA

Weights saved to emotion_cnn_weights.npz
Loaded in at runtime for CPU inference

SECONDARY TARGET

Live feed image preprocessing

Parallelize all per-pixel image processing

Every operation is independent per pixel: ideal SIMD workload

07

CUDA Kernel Implementation

```
// One element per thread — O(n) → O(1) per element
extern "C" __global__
void relu_forward(const float* x,
                  float* out,
                  const int size) {

  int i = blockIdx.x * blockDim.x
        + threadIdx.x;

  if (i < size) {
    float v = x[i];
    out[i] = v > 0.0f ? v : 0.0f;
  }
}
```

SRC/GPU/KERNELS.PY · _RELU_SOURCE · VIA CP.RAWKERNEL

INTERFACE

CuPy 12 RawKernel

LAUNCH CONFIG

256 threads/block grid = [N ÷ 256]

PATTERN

1 output element per thread

Bounds-checked with if (i < size)

Apply operation

08

Model Training Speedup

9.07×

20 EPOCHS

FULL 28K TRAIN SET

IDENTICAL HYPERPARAMETERS

- Each convolutional layer's matrix operations are embarrassingly parallel, thousands of output elements computed simultaneously instead of sequentially

- Backward pass gradients are independent per weight, ideal for SIMT; CPU must compute them one at a time

09

Model Accuracy

TEST ACCURACY · CPU VS GPU

CLASS	CPU	GPU	Δ
Positive	0.750	**0.822**	+0.072
Neutral	0.537	**0.649**	+0.112
Negative	**0.614**	0.555	-0.059
Overall	0.648	**0.694**	**+0.046**

TRAIN VS TEST GAP (OVERFITTING INDICATOR)

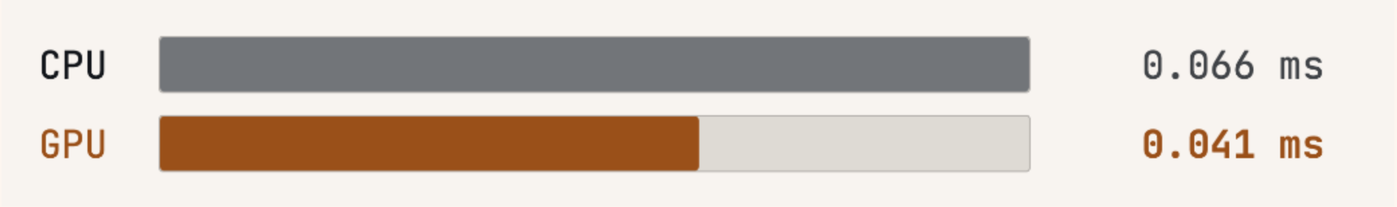
	TRAIN	TEST	GAP
CPU	0.781	0.648	0.133
GPU	0.713	0.694	**0.019**

Same architecture, same hyperparameters. GPU training reached more useful epochs before overfitting.

Live Feed Preprocessing Speedup

1.61×

AVG OVER 50 CONSECUTIVE FRAMES



- 48×48 workload too small to justify kernel-launch overhead and transfer time.
- Higher resolution images or larger data would yield significantly greater speedup.
- **Keeping preprocessing on CPU means the app runs on any machine; the GPU's role ends at training.**

Lessons Learned & Future Work

L · 01

Understand where to and where not to launch kernels.

Identify which operations net benefit from their parallelism. Launch and data transfer time is always part of the cost.

L · 02

Train on GPU, serve from CPU.

GPU computes weights once. CPU loads and serves at interactive speed. Allows application use on any device (No GPU required).

L · 03

Workload size sets the verdict.

The same kernel that loses at 48×48 wins decisively at 256×256.

N · 01

Model Accuracy Improvement

SVM / ViT on the same dataset; different hyperparameters using same model; higher count of more precisely labeled images into dataset.

N · 02

Multi-Face Prediction

Make inference on multiple faces in one frame, using a multi-threaded CPU inference call using pre-computed and loaded in GPU weights.

Understand and implement parallelism where it pays off. Serve it where the user feels it.

AUTHORS
Spencer Scherger · Griffin Kuchar